

# Exploring Object-Oriented Principles in Deep Learning Frameworks: PyTorch vs. TensorFlow

Yutong Zhang, Xujia Zhang

## Abstract

This project analyzes how object-oriented principles appear in two major deep learning frameworks, PyTorch and TensorFlow/Keras. It compares their architectural choices through examples of CNN, optimization, and dataset design. The study finds that PyTorch emphasizes explicit composition and user control through an imperative style, while TensorFlow favors encapsulation and automation via declarative APIs. By mapping these frameworks to classical design patterns—such as Template Method, Strategy, and Facade—the work shows how object-oriented abstraction supports modularity, flexibility, and reusability in machine learning systems. All code and writeup is stored in this repo [https://github.com/XUJIAZHANG2002/CSCI5448\\_project](https://github.com/XUJIAZHANG2002/CSCI5448_project)

## Introduction

PyTorch (Paszke et al. 2019) and TensorFlow (Abadi et al. 2016) are two of the most widely used deep learning frameworks today (Wikipedia Pytorch 2025)(Wikipedia Tensorflow 2025). Both provide high-level tools for building, training, and deploying neural networks, but they differ in how they are designed and how developers interact with them.

This project explores how these frameworks apply object-oriented (OO) principles in their architecture and programming interface. In particular, we examine how core OO concepts, such as encapsulation, inheritance, abstraction, polymorphism, and composition, appear in each system. We focus on the design of `nn.Module` in PyTorch (PyTorch official documents 2024) and `tf.keras.Model` in TensorFlow (Keras official documents 2023), which are the main abstractions for defining neural network models.

By analyzing these frameworks, we aim to understand how OO principles and design patterns support flexibility, modularity, and code reuse in machine learning software. Comparing PyTorch and TensorFlow from this perspective helps us to see how each framework structures its internal design and how users benefit from these abstractions when developing machine learning applications.

## Background: Object-Oriented Design Patterns In Machine Learning

Object-oriented design patterns provide reusable solutions for recurring architectural problems and have become increasingly relevant in modern machine learning systems. Classical foundations such as the "Gang of Four" catalog formalized the core behavioral, structural, and creational patterns that remain applicable across domains (Gamma et al. 1994). Recent works extend these principles to machine learning and deep learning pipelines, emphasizing maintainability, modularity, and human-in-the-loop system integration (Andersen and Maalej 2023)(Chen et al. 2024). Industrial design guides such as *Machine Learning Design Patterns* highlight their practical use for model lifecycle management and reproducibility (Brown, Suryadinata, and others 2023). Together, these works illustrate how traditional object-oriented abstraction and composition principles continue to inform the engineering of large-scale deep learning frameworks.

## Comparing PyTorch & TensorFlow

In this section, we examine two implementations of a convolutional neural network (CNN), a classic deep learning architecture widely used in computer vision. Both versions share the same structure of convolution, pooling, and fully connected layers but differ in their programming paradigms: the PyTorch Model adopts an *imperative* style where computation follows PyTorch control flow, while the TensorFlow Model uses a more *declarative* design built from predefined layer objects and executed within the Keras training loop. We also include an example of loading data in both frameworks, PyTorch Dataset and TensorFlow Dataset, to illustrate how each framework prepares the dataset for the neural network training. These comparisons highlight the object-oriented principles shared by both frameworks, encapsulation of layers and modular composition, as well as their key design differences.

## Model Design

**Layer Composition** Both frameworks rely on classical inheritance and polymorphism to define model hierarchies and enable method overriding. In PyTorch, all layers inherit from `nn.Module`, forming a transparent inheritance chain

(e.g., `Conv2d`  $\rightarrow$  `_ConvNd`  $\rightarrow$  `Module`) that unifies parameter registration, device management, and the polymorphic `forward()` interface. Each subclass customizes its behavior by overriding this method, allowing dynamic and Pythonic execution.

TensorFlow follows a similar structure, where layers such as `tf.keras.layers.Conv2D` inherit from `tf.keras.layers.Layer`, which in turn extends `tf.keras.Model`'s base functionality. The core polymorphic method `call()` enables subclass specialization, but execution is typically constrained by TensorFlow's computational graph tracing.

Thus, both frameworks achieve code reuse through inheritance and runtime polymorphism, yet PyTorch emphasizes flexibility and imperative control, while TensorFlow enforces encapsulation and graph-oriented consistency.

**Activation Function Coupling** TensorFlow couples activations tightly within layer definitions, as seen in `Dense(128, activation='relu')`. This design favors encapsulation and simplicity but violates the Single Responsibility Principle: a single class performs both linear transformation and non-linear activation. It also reduces extensibility, since changing the activation requires reinstantiating the layer. PyTorch, on the contrary, treats activations as separate components (`F.relu()`, `F.softmax()`), promoting loose coupling and composition. Each module has a single responsibility that enables flexible reordering, conditional execution, or custom activation insertion. Thus, TensorFlow prioritizes abstraction, while PyTorch adheres more closely to composition and the Open/Closed Principle.

## Optimization Design

**TensorFlow Optimization** In TensorFlow, model optimization is managed declaratively through the `compile()` and `fit()` interfaces:

```
model.compile(optimizer='adam', loss='
    sparse_categorical_crossentropy')
model.fit(x_train, y_train, epochs=3)
```

Listing 1: TensorFlow optimization example

Here, the optimizer and the loss function are configured at a high level, while the `fit` // method internally performs all steps of the training algorithm: forward propagation, loss computation, gradient calculation, parameter updates, and metric recording. The user does not directly manipulate any of these stages.

This design reflects two core object-oriented patterns:

- **Template Method Pattern:** The `fit()` method defines the *skeleton* of the training algorithm, while subclasses override `call()` to specify the forward pass. This enforces a consistent workflow while allowing model-specific customization.
- **Facade Pattern:** The `Model` class acts as a high-level facade, hiding multiple interacting subsystems—optimizers, losses, metrics, callbacks, and data pipelines—behind a single, simplified interface (`fit()`). This abstraction improves usability and encapsulation,

enabling users to train complex models with a single method call.

By combining these two patterns, TensorFlow achieves a strongly encapsulated, framework-driven optimization process: users define *what* to optimize, while the framework determines *how* optimization is executed.

**PyTorch Optimization** PyTorch exposes optimization as a user-managed process that favors flexibility and explicit control:

```
optimizer = torch.optim.Adam(model.
    parameters())

for x, y in dataloader:
    optimizer.zero_grad()
    output = model(x)
    loss = criterion(output, y)
    loss.backward()
    optimizer.step()
```

Listing 2: PyTorch optimization example

Here, the user implements the entire training loop: forward pass, loss evaluation, gradient computation, and parameter updates. This design exemplifies **composition over inheritance**—the optimizer and loss function are composed into the loop as independent, interchangeable strategy objects. It offers greater flexibility and transparency at the cost of additional user responsibility.

**Single Responsibility Principle** TensorFlow violates the Single Responsibility Principle in its optimization design: the `Model` class not only defines the network architecture but also manages the optimizer, loss computation, training loop, gradient updates, and callbacks through `fit()`. This centralization couples multiple responsibilities into a single class.

In contrast, PyTorch follows SRP by separating concerns clearly. The `Module` class handles only the forward computation, the `Optimizer` updates parameters, and the training loop orchestrates these components explicitly. Each object has a single, well-defined purpose, making the design more modular, testable, and extendable.

## Dataset and DataLoader Design

TensorFlow's `tf.data.Dataset` follows a declarative and encapsulated design. It combines multiple responsibilities—data access, batching, and parallelization—within a single class, exposing a fluent interface through method chaining (`.map()`, `.batch()`, `.prefetch()`). This design is an example of the *Builder* and *Template Method* patterns: users construct data pipelines step by step by chaining methods, while the framework encapsulates the internal logic for execution. The builder-style interface provides high abstraction and ease of use, but it also introduces tight coupling between the data pipeline and TensorFlow's execution graph, limiting extensibility and user control.

In contrast, PyTorch adopts an explicit, compositional architecture built on the *Strategy* and *Iterator* patterns. The `Dataset`, `Sampler`, and `DataLoader` classes act as

independent yet cooperating components that adhere to the Single Responsibility and Open/Closed Principles. The `Dataset` defines how samples are accessed, the `Sampler` determines the order in which samples are drawn, and the `DataLoader` manages batching, shuffling, and parallel loading. These components interact through the Iterator pattern, exposing a uniform interface (for `batch` in `DataLoader: ...`) that abstracts away the underlying data sources. This modular structure enables high extensibility—users can easily replace or extend any strategy independently—while maintaining a consistent data access interface. The trade-off is that users must explicitly compose and manage these components, gaining flexibility at the cost of higher control overhead.

```
from torch.utils.data import Dataset,
    DataLoader
import torch

# Custom dataset
class MyDataset(Dataset):
    def __init__(self, data, labels):
        self.data = data
        self.labels = labels
    def __getitem__(self, idx):
        return self.data[idx], self.labels[idx]
    def __len__(self):
        return len(self.data)

# Composition: Dataset + DataLoader
dataset = MyDataset(torch.randn(1000, 32),
    torch.randint(0, 10, (1000,)))
loader = DataLoader(dataset, batch_size=32,
    shuffle=True)

# Iteration via Iterator pattern
for batch_x, batch_y in loader:
    print(batch_x.shape, batch_y.shape)
```

Listing 3: PyTorch dataset and dataloader example

Here, `DataLoader` acts as the context that orchestrates data access through interchangeable sampling and batching strategies, while the `Dataset` and `Sampler` define independent behaviors. This composition-based design exemplifies the Strategy pattern and promotes modular reusability—contrasting with TensorFlow’s tightly encapsulated, builder-style data pipeline.

## Similarities in OO Network Definition Across ResNet and Transformer

We examine two representative architectures commonly used in deep learning: ResNet-50 ((He et al. 2015)) and the Transformer ((Vaswani et al. 2023)) encoder. ResNet-50 is a deep convolutional network composed of stacked residual bottleneck blocks, each of which applies a sequence of convolutions and then adds the input back to the output. This residual connection enables the network to train effectively at depths that were previously unstable, making ResNet a foundational model for image recognition. The Transformer

is a sequence-processing architecture built around multi-head self-attention and position-wise feed-forward blocks. Instead of spatial convolutions, it models relationships between all pairs of tokens, allowing it to capture long-range dependencies with high flexibility. Although these architectures differ significantly in purpose and internal computation, both are constructed from modular computational blocks and therefore provide ideal case studies for analyzing the object-oriented design philosophies of PyTorch and TensorFlow. Both the ResNet-50 and Transformer examples reveal strong structural similarities in the way PyTorch and TensorFlow define networks using object-oriented abstractions. In both frameworks, a model is built from modular building blocks, each represented as a class (`nn.Module` or `tf.keras.layers.Layer`) with learnable parameters and a single forward transformation.

For example, the Bottleneck block in PyTorch:

```
class Bottleneck(nn.Module):
    def __init__(self, in_planes, planes,
        stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_planes,
            planes, 1)
        ...
    def forward(self, x):
        out = self.conv1(x)
        ...
        return F.relu(out + identity)
```

corresponds closely to the Keras implementation:

```
class Bottleneck(layers.Layer):
    def __init__(self, planes, stride=1):
        super().__init__()
        self.conv1 = layers.Conv2D(planes,
            1)
        ...
    def call(self, x):
        out = self.conv1(x)
        ...
        return tf.nn.relu(out + identity)
```

The same pattern appears in the Transformer examples. A PyTorch multi-head attention layer:

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        self.q = nn.Linear(d_model, d_model)
        ...
```

mirrors its TensorFlow counterpart:

```
class TransformerBlock(layers.Layer):
    def __init__(self, d_model, num_heads):
        super().__init__()
        self.mha = layers.MultiHeadAttention(
            num_heads, ...)
        ...
```

In both frameworks, networks are formed by composing these blocks into ordered stages or layers:

```
# PyTorch ResNet stage
self.layer3 = nn.Sequential(
    Bottleneck(...), Bottleneck(...), ...
)

# PyTorch Transformer layers
for blk in self.layers:
    x = blk(x)
```

and equivalently in TensorFlow:

```
# TensorFlow ResNet stage
for block in self.layer3:
    x = block(x)

# TensorFlow Transformer
for layer in self.layers_t:
    x = layer(x)
```

## Differences in OO Network Definition Across ResNet-50 and Transformer

While the overall structure of ResNet-50 and the Transformer appears similar between PyTorch and TensorFlow, the two frameworks diverge in their object-oriented design philosophy. These differences become especially visible when examining larger portions of code. In PyTorch, the Bottleneck block in ResNet-50 is implemented with completely explicit control flow:

```
def forward(self, x):
    identity = x
    out = self.relu(self.bn1(self.conv1(x)))
    out = self.relu(self.bn2(self.conv2(out)))
    out = self.bn3(self.conv3(out))
    if self.downsample:
        identity = self.downsample(x)
    return self.relu(out + identity)
```

Every operation, such as convolution, normalization, activation, skip path, and addition, is explained inside the `forward()` method. This structure embodies minimal encapsulation and directly exposes the computational graph. The advantage of this design is complete transparency: developers can alter the residual path, add conditional execution, or insert new components without getting troubled dealing with abstraction. However, the downside is increased verbosity and a stronger dependence on user discipline, since PyTorch provides fewer framework-level guarantees about execution semantics.

In TensorFlow/Keras, the equivalent block is written with similar surface structure, but the surrounding OO environment is significantly more encapsulated:

```
def call(self, x):
    identity = x
    out = tf.nn.relu(self.bn1(self.conv1(x)))
    out = tf.nn.relu(self.bn2(self.conv2(out)))
    out = self.bn3(self.conv3(out))
```

```
if self.shortcut:
    identity = self.shortcut(x)
return tf.nn.relu(out + identity)
```

Although the code appears nearly identical, the operations inside `conv1`, `bn1`, and `shortcut` trigger substantial framework-managed behavior: variable creation, shape inference, mixed-precision policies, and graph tracing. Keras layers therefore behave as higher-level OO components. The benefit is cleaner abstraction, portability, and less likely to make mistakes, but the cost is reduced modifiability. Altering internal execution semantics often requires subclassing or bypassing the framework.

These differences become more obvious in the Transformer examples. PyTorch constructs multi-head attention manually:

```
Q = self.q_proj(x).view(B, T, H, D)
K = self.k_proj(x).view(...)
scores = Q @ K.transpose(-2, -1)
attn = F.softmax(scores, dim=-1)
out = self.out_proj(attn @ V)
```

Here, the attention mechanism is expressed entirely as tensor operations. PyTorch encourages a bottom-up, compositional approach. This design provides extraordinary flexibility: coders can replace the softmax with alternative kernels, implement routing schemes, or introduce sparse attention. The drawback is additional implementation burden.

TensorFlow replaces this logic with a single encapsulated layer:

```
attn = self.mha(x, x)
# where self.mha is defined as
self.mha = layers.MultiHeadAttention(
    num_heads, ...)
```

The `MultiHeadAttention` layer internally performs projection, reshaping, scaling, masking, and concatenation. From an OO perspective, this is a strong example of the *Facade Pattern*: multiple subsystems are hidden behind a single interface. The benefit is conciseness, consistency, and reduced risk of implementation error. The limitation is restricted control: modifying internal components of `mha` requires reimplementing the entire layer or dropping into lower-level APIs.

Across both architectures, the core OO difference is therefore the level of encapsulation. PyTorch treats each mathematical step as part of the model definition and exposes it in code. TensorFlow treats each architectural concept as a reusable object whose internal management is delegated to the framework. PyTorch is more explicit, flexible, and research-oriented, while TensorFlow is more abstract, stable, and system-oriented. These differences reflect opposing philosophies of OO design: PyTorch favors explicit composition with minimal abstraction, whereas TensorFlow favors declarative modeling with strong framework-managed behavior.

## Design Trade-offs and Practical Implications

The differences observed throughout this study, such as spanning layer composition, activation design, optimization,

dataset construction, and full network definition, point to two fundamentally different OO philosophies embodied by PyTorch and TensorFlow. These trade-offs become especially clear when grounded in the concrete examples of ResNet-50 and the Transformer encoder.

Across all earlier sections, PyTorch consistently favors explicit composition and developer control. In both the ResNet-50 Bottleneck block and the Transformer attention module, every computation is written directly inside the `forward()` method. The skip connection, residual addition, projection layers, and even the attention softmax are constructed step by step. The same pattern appears in PyTorch’s optimizer usage, where the training loop is manually implemented, and in its data pipeline, where `Dataset`, `Sampler`, and `DataLoader` are assembled as independent strategy objects. The practical implication is that PyTorch excels when the model’s internal behavior must be modified or experimented with: code designers can edit the residual path in ResNet-50, swap out attention computations, or introduce dynamic, data-dependent control flow with minimal resistance from the framework. The cost is that PyTorch provides few guardrails; correctness, shape management, and execution semantics are the responsibility of the developer.

TensorFlow, in contrast, follows a more encapsulated and declarative OO approach. In the Keras Bottleneck block, the visible code appears similar to PyTorch, but each layer object hides substantial internal behavior: variable initialization, shape inference, mixed precision policies, and graph construction. The Transformer example makes this contrast even sharper: multi-head attention collapses into a single `MultiHeadAttention` layer, a facade that bundles projection, scaling, masking, and concatenation. The same pattern applies to TensorFlow’s optimizer model, where `compile()` and `fit()` define the training algorithm, and to its `tf.data` pipeline, where data transformations are declared rather than coded explicitly. The practical benefit is reliability and strong framework-managed consistency: models defined in Keras inherit predictable behavior across devices, precisions, and deployment environments. The trade-off is reduced extensibility; altering a core component such as the attention kernel or the dataflow order typically requires reimplementing internal subsystems rather than locally modifying a block.

Taken together, these differences highlight a fundamental trade-off in OO design for deep learning frameworks. PyTorch’s imperative, low-encapsulation approach offers maximal expressiveness and suits experimental research where the model definition itself is subject to change. TensorFlow’s encapsulated, framework-driven architecture favors stability, portability, and large-scale deployment, where predictable behavior across platforms outweighs fine-grained control. These trade-offs directly reflect how each framework maps classical OO patterns, such as Composite, Strategy, Builder, and Facade, to the practical demands of deep learning. Understanding these design choices enables developers to select the appropriate framework for their needs and clarifies why PyTorch tends to dominate research while TensorFlow remains popular in production settings.

## Why Researchers Moved Toward PyTorch

The shift from TensorFlow to PyTorch in the research community can be explained by the OO design differences observed in our ResNet-50 and Transformer examples. PyTorch defines networks as ordinary Python objects whose internal behavior is fully exposed in the `forward()` method. Every operation in a ResNet bottleneck block or in multi-head attention—projection, reshaping, residual connections, attention scoring—is written explicitly, allowing researchers to modify or replace components with minimal friction. This openness to modification is essential when exploring new architectures whose internal structure frequently changes.

TensorFlow’s design is more encapsulated. Components such as `Conv2D`, `BatchNormalization`, and especially `MultiHeadAttention` hide large amounts of behavior inside framework-managed objects. While this improves reliability and portability, it reduces the degree to which developers can alter low-level mechanisms. Redefining an internal attention step or experimenting with a non-standard residual path often requires subclassing or reimplementing entire layers, creating structural resistance to architectural innovation.

As models like Transformers, graph networks, and large-scale sequence systems grew in complexity, this difference became decisive. PyTorch’s transparent, imperative OO style allowed researchers to treat model definition as flexible code rather than as a constrained declarative graph. Consequently, PyTorch became the preferred framework for rapid experimentation, making it the dominant choice for modern academic deep learning research.

## Conclusion

PyTorch and TensorFlow both implement core object-oriented ideas but embody distinct philosophies. PyTorch’s design promotes clarity and flexibility through explicit control of training and data flow, aligning with composition and single-responsibility principles. TensorFlow, in contrast, abstracts complexity into unified interfaces that enhance usability but reduce transparency. Understanding these trade-offs illustrates how software design patterns shape the balance between control and abstraction in modern deep learning frameworks.

## References

- [Abadi et al. 2016] Abadi, M.; Barham, P.; Chen, J.; Chen, Z.; Davis, A.; Dean, J.; Devin, M.; Ghemawat, S.; Irving, G.; Isard, M.; et al. 2016. Tensorflow: Large-scale machine learning on heterogeneous distributed systems. *arXiv preprint arXiv:1603.04467*.
- [Andersen and Maalej 2023] Andersen, J. S., and Maalej, W. 2023. Design patterns for machine learning based systems with human-in-the-loop. *IEEE Software*.
- [Brown, Suryadinata, and others 2023] Brown, V.; Suryadinata, S.; et al. 2023. *Machine Learning Design Patterns*. O’Reilly Media.
- [Chen et al. 2024] Chen, K.; Bi, Z.; Wang, T.; Wen, Y.; et al. 2024. Deep learning and machine learning: Advancing big

data analytics and management with design patterns. *arXiv preprint arXiv:2409.19916*.

[Gamma et al. 1994] Gamma, E.; Helm, R.; Johnson, R.; and Vlissides, J. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

[He et al. 2015] He, K.; Zhang, X.; Ren, S.; and Sun, J. 2015. Deep residual learning for image recognition.

[Keras official documents 2023] Keras official documents. 2023. Keras: The high-level api for tensorflow. <https://www.tensorflow.org/guide/keras>. Accessed: 2025-11-08.

[Paszke et al. 2019] Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J.; Chanan, G.; Killeen, T.; Lin, Z.; Gimelshein, N.; Antiga, L.; Desmaison, A.; Kopf, A.; Yang, E.; DeVito, Z.; Raison, M.; Tejani, A.; Chilamkurthy, S.; Steiner, B.; Fang, L.; Bai, J.; and Chintala, S. 2019. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems* 32.

[PyTorch official documents 2024] PyTorch official documents. 2024. Pytorch documentation. <https://pytorch.org/docs/stable/>. Accessed: 2025-11-08.

[Vaswani et al. 2023] Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, L.; and Polosukhin, I. 2023. Attention is all you need.

[Wikipedia Pytorch 2025] Wikipedia Pytorch. 2025. PyTorch — Wikipedia. <https://en.wikipedia.org/wiki/PyTorch>. Accessed: 2025-11-08.

[Wikipedia Tensorflow 2025] Wikipedia Tensorflow. 2025. TensorFlow — Wikipedia. <https://en.wikipedia.org/wiki/TensorFlow>. Accessed: 2025-11-08.